서 울 대 학 교
SEOUL NATIONAL UNIVERSITY

Systems Software & Architecture Laboratory
Dept. of Computer Science and Engineering

4190.307: Operating Systems
Midterm Exam. (Fall 2025)

12:30 – 13:45, October 21, 2025. Instructor: Jin-Soo Kim

Student ID: _____

Name: _____

Codename: _____

Q1 (40)		Q4 (50)	
Q2 (70)		Q5 (70)	
Q3 (30)			
		Total (260)	

I hereby certify that I have neither given nor received aid in answering the questions on this exam. _____
(Signature)

1. Fill in the blank with a single English word (including hyphenated ones). (4 points each)

(1) The OS takes a(an) ( ) resource (such as the processor, or memory, or a disk) and transforms it into a more general, powerful, and easy-to-use virtual form of itself.

(2) Another early name for the OS was the ( ) or even the master control program.

(3) To execute a system call, a program must execute a special ( ) instruction.

(4) ( ) execution has the obvious advantages of being fast; the program runs natively on the hardware CPU and thus executes as quickly as one would expect.

(5) The addition of a(an) ( ) interrupt gives the OS the ability to run again on a CPU even if processes act in a non-cooperative fashion. Thus, this hardware feature is essential in helping the OS maintain control of the machine.

(6) In early operating systems, the loading process is done ( ), i.e., all at once before running the program; modern OSes perform the process lazily, i.e., by loading pieces of code or data only as they are needed during program execution.

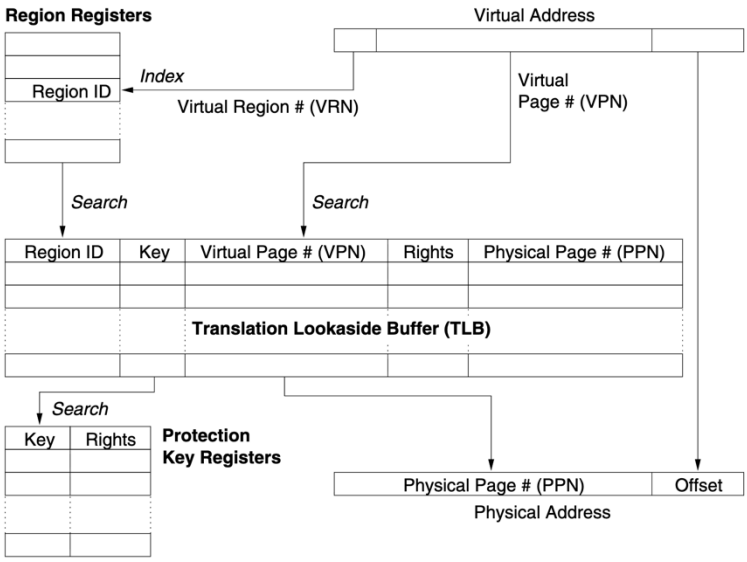
(7) A scheduler, for example, may optimize performance but at the cost of preventing a few jobs from running, thus decreasing ( ). This conundrum shows us that life isn't always perfect.

(8) As the operating system rarely knows what is best for each and every process of the system, it is often useful to provide interfaces to allow users or administrators to provide some ( ) to the OS.

(9) One major goal of a virtual memory system is transparency. The OS should implement virtual memory in a way that is ( ) to the running program.

(10) There are two types of locality that programs tend to exhibit. The first is known as ( ) locality, which states that if a page P is accessed, it is likely the pages around it (say P - 1 or P + 1) will also likely be accessed.

2. Introduced in 2000, Itanium (IA-64) was Intel and HP's ambitious 64-bit architecture for the next decades, but it is now remembered as an unsuccessful architecture. Itanium's memory-management unit (MMU) had some unusual features. The following figure briefly shows the address translation process in the Itanium architecture.



The top three bits of a 64-bit virtual address selects one of eight virtual regions, and the remaining 61 bits provide the virtual page number (VPN) and the page offset. Itanium supports a wide range of page sizes—4KB, 8KB, 16KB, 64KB, 256KB, 1MB, 4MB, 16MB, 256MB, …, up to 4GB. Each region has an associated Region Register that supplies a Region ID (RID) and a preferred page size. The TLB is indexed by the pair <RID, VPN>.

Itanium has hardware support for filling the TLB by walking a page table called the *virtual hashed page table* (VHPT). There are two hardware supported page-table formats: short-format and long-format VHPT. The short-format VHPT is a linear virtual array page table that is indexed by the VPN and maps a single region, hence up to eight are required per process. It is compact (8 bytes/PTE) and assumes the region's preferred page size from Region Register. The long-format VHPT is a proper hashed page table, indexed by a hash of the VPN. A single table can be used for all regions, where each PTE is 32 bytes containing the explicit page size, protection key, access rights, attributes, a hash tag, etc.

(1) What are the differences between *segments* (in traditional segmentation-based architectures such as Multics) and *regions* in the Itanium architecture? (10 points)

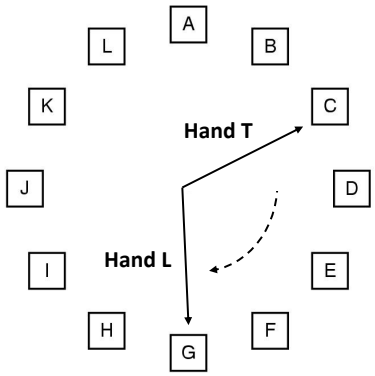
(2) What are the benefits of using Region ID (RID) for TLB? (10 points)

(3) What is the maximum size of the short-format VHPT for a single region when the page size is 4KB? (10 points)

(4) Assume each region uses a multi-level page table instead of the short-format VHPT (with the same PTE format). If each page table must fit within a single page, how many levels are required when the page size is 32 KB? Explain why. (10 points)

(5) Assume the system is configured to use the long-format VHPT. The long-format VHPT is a hash table for fast TLB refills; it is neither required nor practical for it to contain all PTEs. On a TLB miss, the hardware walker probes the VHPT by hashing <RID, VPN>. If no valid entry is found in VHPT, the processor traps into the kernel's software TLB miss handler, which looks up the mapping in the kernel's data structures, inserts a TLB entry, and may also populate/update the VHPT. List all the possible cases the TLB miss handler must cover and explain how each is handled. (30 points)

3. In the traditional (one-handed) clock algorithm, all page frames are arranged in a circular list, and a single "hand" sweeps around it. When the hand encounters a page whose reference bit is set, it clears the bit and moves on. Otherwise, the page frame is chosen as a victim. Dirty pages are handled lazily; if the chosen victim is clean, it can be reclaimed immediately. If it's dirty, the system initiates a write-back and continues scanning for other victims.



The 4.3BSD UNIX is known to use a page replacement algorithm called **two-handed clock**. In the two-handed clock algorithm, there are two hands separated by a fixed gap around the same circular list. The leading hand (Hand L in figure) clears reference bits and, for dirty pages, triggers write-back so cleaning can proceed in the background. The trailing hand (Hand T in figure) follows behind after that gap and reclaims any page frame whose reference bit remains clear and is either clean or has finished write-back when it arrives. Answer the following questions.

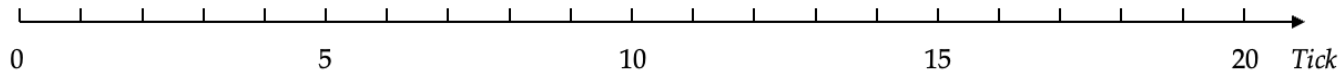
(1) What are the advantages of the two-handed clock algorithm over the (one-handed) clock? (15 points)

(2) Describe one advantage and one disadvantage of using a larger gap between the leading and trailing hands. (15 points)

4. After surviving the operating systems course, a group of students developed SnuOS, where processes have just five nice levels: {-2, -1, 0, +1, +2} (default: 0) – because they never figured out why UNIX needs forty. Still, they decided to follow the UNIX tradition that lower nice values mean higher priority. We consider three CPU-bound processes A, B, and C with nice values 0, +1, and -1, respectively. Answer the following questions.

(1) The SnuOS-v1 employs a round-robin scheduler that grants larger time slices to higher-priority processes. A process with nice n receives a time slice, $T(n) = 16 \times \left(\frac{1}{2}\right)^{(n+2)} ticks$. Processes are scheduled in order of arrival (FCFS among ready tasks). Once dispatched, a process performs no I/O and runs until its time slice expires (no preemption by new processes). Suppose processes A, B, and C arrive at ticks 0, 5, and 7, respectively.

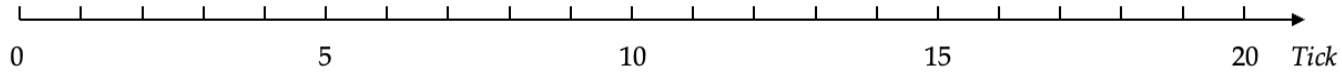
A. Complete the following diagram that shows the scheduling sequence from ticks 0 to 20. (10 points)



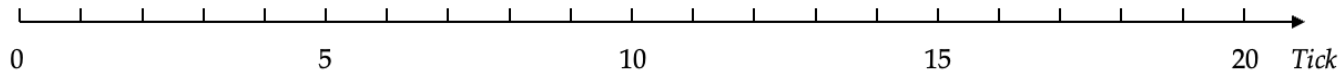
B. What is the response time for process B in ticks? (5 points)

(2) The SnuOS-v2 moves on to the preemptive priority scheduler, where the time slice is fixed to 4 ticks for all processes.

A. Complete the schedule from ticks 0 to 20, assuming processes A, B, and C all arrive at tick 0. (10 points)



B. Complete the schedule when processes A, B, and C all arrive at tick 0, but C blocks for I/O at tick 6 and resumes at tick 10. (10 points)



(3) Unsatisfied with the earlier schedulers, the SnuOS-v3 developers built SFS (SNU Fair Share), a simplified scheduler inspired by Linux CFS. As in CFS, each nice value maps to a weight that determines a process's share of CPU time: $W(n) = 400 \times \left(\frac{1}{2}\right)^{(n+2)}$.

A. When A, B, and C are all runnable, compute the fraction of CPU each should receive in steady state. (5 points)

B. Recompute the CPU shares for A and C after B terminates. (5 points)

C. What is C's virtual runtime after C runs for 30 ms? (5 points)

5. Answer the following questions on xv6 and PA2. For True/False questions, +5 points for right answer, -5 points for wrong answer, and 0 points for no answer.

(1) The system call number is passed in **`a0`** register. (True / False)

(2) By default, timer interrupts are always taken in the machine mode. (True / False)

(3) Assume the machine-mode trap handler is invoked only by machine software interrupts. What should be the first instruction? Show the assembly instruction and explain why. (10 points)

(4) If the machine-mode trap handler is invoked only by the **`ecall`** instruction from the supervisor mode, do we still need the instruction shown in the previous question? Explain why. (10 points)

(5) Describe briefly what you had to do inside **`ipi_init()`** for PA2. (20 points)

(6) Assume that the **`pong()`** implementation works correctly and you are implementing the **`ping()`** system call. You have verified that the machine-mode interrupt handler is invoked properly. However, the **`ping()`** system call never returns to user space—it keeps re-entering M-mode continuously. What mistake could cause this repeated trap into M-mode? (10 points)

(7) When a hart receives a machine software interrupt, what happens if it does not clear its **`msip`** register in CLINT to 0? (10 points)